

# Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud

Arnar Birgisson  
Chalmers University of Technology  
arnarbi@gmail.com

Joe Gibbs Politz  
Brown University  
joe@cs.brown.edu

Úlfar Erlingsson, Ankur Taly,  
Michael Vrabie, and Mark Lentczner  
Google Inc  
{ulfar,ataly,mvrabie,mzero}@google.com

**Abstract**—Controlled sharing is fundamental to distributed systems; yet, on the Web, and in the Cloud, sharing is still based on rudimentary mechanisms. More flexible, decentralized cryptographic authorization credentials have not been adopted, largely because their mechanisms have not been incrementally deployable, simple enough, or efficient enough to implement across the relevant systems and devices.

This paper introduces *macaroons*: flexible authorization credentials for Cloud services that support *decentralized delegation* between principals. Macaroons are based on a construction that uses nested, chained MACs (e.g., HMACs [43]) in a manner that is highly efficient, easy to deploy, and widely applicable.

Although macaroons are bearer credentials, like Web cookies, macaroons embed *caveats* that *attenuate* and *contextually confine* when, where, by who, and for what purpose a target service should authorize requests. This paper describes macaroons and motivates their design, compares them to other credential systems, such as cookies and SPKI/SDSI [14], evaluates and measures a prototype implementation, and discusses practical security and application considerations. In particular, it is considered how macaroons can enable more fine-grained authorization in the Cloud, e.g., by strengthening mechanisms like OAuth2 [17], and a formalization of macaroons is given in authorization logic.

## I. INTRODUCTION

*Macaroons* are authorization credentials that provide flexible support for controlled sharing in decentralized, distributed systems. Macaroons are widely applicable since they are a form of bearer credentials—much like commonly-used cookies on the Web—and have an efficient construction based on keyed cryptographic message digests [43].

Macaroons are designed for the Web, mobile devices, and the related distributed systems collectively known as the *Cloud*. Such modern software is often constructed as a decentralized graph of collaborative, loosely-coupled services. Those services comprise different protection domains, communication channels, execution environments, and implementations—with each service reflecting the characteristics and interests of the different underlying stakeholders. Thus, security and access

control are of critical concern, especially as the Cloud is commonly used for sharing private, sensitive end-user data, e.g., through email or social networking applications.

Unfortunately, controlled sharing in the Cloud is founded on basic, rudimentary authorization mechanisms, such as HTTP cookies that carry pure bearer tokens [21, 54]. Thus, today, it is practically impossible for the owner of a private, sensitive image stored at one Cloud service to email a URL link to that image, safely—given the many opportunities for impersonation and eavesdropping—such that the image can be seen only by logged-in members of a group of users that the owner maintains at another, unrelated Cloud service. Currently, this use case is possible only if the image, access group, and users are all at a single service, or if two Cloud services keep special, pairwise ties using custom, proprietary mechanisms (e.g., as done by Dropbox and Facebook [55]).

Of course, the ubiquitous use of bearer tokens is due to advantages—such as simplicity and ease of adoption—that cannot be overlooked. For example, bearer tokens can easily authorize access for unregistered users (e.g., to the shopping cart of a first-time visitor to a Cloud service) or from unnamed, transient contexts (e.g., from a pop-up window shown during private, incognito Web browsing). Such dynamic and short-lived principals arise naturally in distributed systems, like the Cloud and the “Grid” [47]. In comparison, most authorization mechanisms based on public-key certificates are not directly suited to the Cloud, since they are based on more expensive primitives that can be difficult to deploy, and define long-lived, linkable identities, which may impact end-user privacy [21].

Even so, the inflexibility of current Cloud authorization is quite unsatisfactory. Most users will have first-hand experience of the resulting frustrations—for example, because they have clicked on a shared URL, only to be redirected to a page requesting account creation or sharing of their existing online identity. Similarly, many users will have uncomfortably surrendered their passwords to use some Cloud service functionality, such as to populate an address book (e.g., on LinkedIn.com) or to aggregate their financial data (e.g., on mint.com).

Macaroons aim to combine the best aspects of using bearer tokens and using flexible, public-key certificates for authorization, by providing (i) the wide applicability, ease-of-use, and privacy benefits of bearer credentials based on fast cryptographic primitives, (ii) the expressiveness of truly decentralized credentials based on authorization logic, like SPKI/SDSI [14], and (iii) general, precise restrictions on how, where, and when credentials may be used.

Macaroons allow authority to be *delegated* between protection domains with both *attenuation* and *contextual confinement*. For this, each macaroon embeds *caveats* which are predicates that restrict the macaroon’s authority, as well as the context in which it may be successfully used. For example, such caveats may attenuate a macaroon by limiting what objects and what actions it permits, or contextually confine it by requiring additional evidence, such as third-party signatures, or by restricting when, from where, or in what other observable context it may be used. In particular, a macaroon that authorizes service requests may contain caveats that require proof that the requests have been audited and approved by an abuse-detection service, and come from a specific device with a particular authenticated user.

In short, Macaroons allow Cloud services to authorize resource access using efficient, restricted bearer tokens that can be delegated further, and, via embedded caveats, attenuated in scope, subjected to third-party inspection and approval, and confined to be used only in certain contexts. Fundamental to this flexibility of macaroons is a chained construction of nested HMAC values that simultaneously provides key distribution and protects integrity, generalizing earlier work dating back to the Amoeba operating system [3, 25, 51].

## II. MOTIVATION AND FOUNDATIONS

Conceptually, macaroons make an assertion about conditional access to a *target service*<sup>1</sup>, along these lines: “The bearer may perform a request *as long as* predicates  $C_1, \dots, C_n$  hold true in the context of that request.”

In distributed systems, the complexity of such assertions can grow surprisingly quickly, even for simple, practical authorization policies. For example, if, as in Section I, the owner of a private image at *TS* wants to safely email its URL link to the members of a group *G* kept at an unrelated service *A*, macaroons may establish an assertion like the following:

The bearer, client *C*, may perform the request at *TS*  
*as long as* the operation is read,  
*as long as* the object is `privateImage.jpg`,  
*as long as* the user at *C* is logged into service *A*,  
*as long as* that logged-in user is in group *G* at *A*,  
*as long as* the above proofs are recent/fresh enough,  
 and, *as long as* the user at *C* didn’t just log out of *A*.

Furthermore, the image may be subject to additional access controls at *TS* (e.g., to track changes of ownership), the request from client *C* may be performed from an unnamed context (e.g., a transient Web-page frame), and the logged-in user’s privacy may need to be protected from *TS*. (Variants of this assertion are the basis for paper’s examples and the end-to-end evaluation in Section VII.)

Due to their flexible, efficient HMAC-based construction, macaroons can enforce such complex assertions despite tight freshness constraints and the need to closely guard privacy.

Macaroons are an instance of what has been termed *claims-based*, *proof-carrying*, or simply *credentials-based authorization*, and is defined precisely in [48, Chapter 9]. Over the last

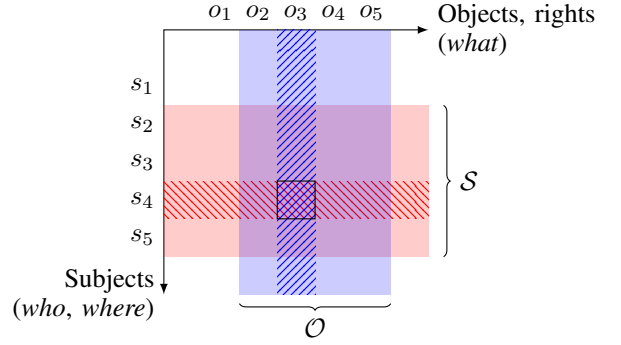


Fig. 1. Macaroons in the Access Control Matrix model [36]. By attenuating a macaroon authorizing  $\mathcal{O}$ , a derived macaroon may permit access only to  $o_3$ . Even when held by all subjects in  $\mathcal{S}$ , contextual caveats may confine the use of this derived macaroon to the observable context of principal  $s_4$ .

two decades, several such credentials-based mechanisms have been developed using public-key certificates and the formalism of authorization logic [37, 53]. Of these, many support highly flexible, decentralized authorization for distributed systems, with notable examples including SPKI/SDSI [14], Trust Management [11], active certificates [12], and SecPAL [9]. However—whether due to the cost of their primitives, their need for federation, coordination, or public identities, or the complexity of their implementation—such flexible public-key-certificate mechanisms have seen little use in the Cloud [54].

Instead, near exclusively, authorization in the Cloud is based on a pattern of pure *bearer tokens*: secrets that grant unconditional authority, and are sent directly between protection domains, even on unencrypted channels [21]. Examples of pure bearer tokens include the HTTP cookies sent to identify authorized sessions, the secrets for API keys, OAuth tokens, and SAML or OpenID assertions used at other protocol layers, as well as the “unlisted URLs” that are commonly created and shared via email or chat to provide limited access to private content [28, 50, 54].

Efficiency, wide applicability, and ease-of-adoption are three main reasons for bearer tokens’ popularity in the Cloud. Macaroons share all three of these properties, in their HMAC-based construction. First, macaroons are simple enough to implement, and fast-enough to use in near any environment—even a severely-limited, out-of-date Web browser executing on slow, resource-constrained mobile hardware. Second, macaroons are *bearer credentials* and, so, principals may gain authority simply by gathering macaroons. Third, macaroons still provide new benefits even when unilaterally adopted, e.g., by allowing users to restrict when, and from where, credentials may be used [21].

By supporting *attenuation*, *delegation*, and *contextual confinement*, macaroons have much greater expressiveness than previously existing Cloud bearer tokens. Thus, as shown in Figure 1, a user that holds access rights to the objects in  $\mathcal{O}$  can safely pass, via email, a macaroon for the right to access a single object  $o_4$ , limited to the specific recipient  $s_4$ , even if all the parties in  $\mathcal{S}$  can read the email. Moreover, as shown in Figure 2, the user can do this without the operational difficulty or latency and bandwidth costs of running an intermediate proxy service, and even without contacting the target service to mint new, restricted credentials.

<sup>1</sup>This section shows key terms and new concepts in *italics* when introduced and informally defined; the next two sections and the appendix provide detailed definitions and describe concrete implementations.

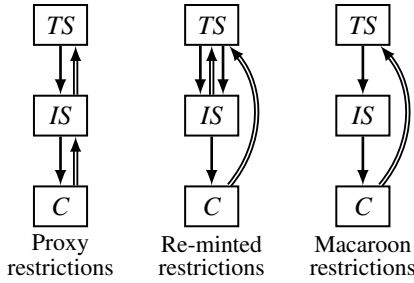


Fig. 2. Three ways for an intermediary service *IS* to give a client *C* limited access to a target service *TS*: by proxying requests, by having *TS* mint new, restricted credentials for *C*, or by *IS* deriving new macaroons for *C*. Arrows show credential passing and doubled arrows indicate access.

In their expressiveness, and in aspects of their construction, macaroons are related to cascaded proxy certificates [44], restricted delegation [33], active certificates [12], proof-carrying authentication [6], and Grid computing [22], as well as systems for limiting resource consumption [23] and access to networked storage services [3, 25]. Indeed, compared to systems based on authorization logic like SPKI/SDSI [14], it must be stated explicitly that macaroons have no real advantage other than their efficiency and ease of adoption. In particular, in terms of functionality and security, HMAC-based macaroons have the clear disadvantage of being verifiable only by the target service, and by revealing to it certain keys, which prevents useful types of key reuse. (This is detailed in Sections IV-D and V-B and Section V-H outlines a public-key variant of macaroons eliminating this disadvantage.) However, considering the macaroons’ efficiency and flexibility, remarkably few such trade-offs must be made for their practical deployment.

In particular, revocation of authority is a long-standing challenge for credentials-based authorization systems, as detailed in [19, Section 6] and [48, Chapter 9.8]. Broadly, this challenge has been addressed by (i) using very short-lived credentials, (ii) allowing the addition of freshness constraints, (iii) relying on external, authoritative state (such as revocation lists, or epoch counters), and (iv) splitting credentials, and granting authority only to a freshly-gathered collection. Macaroons excel at all four of these revocation strategies: due to their efficiency, and flexibility, macaroons may live only for seconds, and may be split or constrained in any manner, as described in the following sections.

#### A. Concepts and Constructions

Macaroons operate from a service’s point-of-view, in that they permit a given *target service*, such as a Cloud resource server<sup>2</sup>, to mint a bearer credential that grants authority to access some aspects of the service. A distinguishing characteristic of macaroon credentials is that their bearer can delegate parts of their authority to other principals by deriving new macaroons that both *attenuate* the accessible aspects of the target service and also restrict, via *contextual confinement*, from where, by who, and with what extra evidence, the derived macaroon may be used.

To restrict the authority of a derived macaroon, *caveats* are added and embedded within it. Each macaroon caveat states a

predicate that must hold true for the context of any requests that the derived macaroon is to authorize at the target service—with that predicate evaluated by the target service in the context of each particular request. Notably, as shown in Figure 2, by adding caveats, new, more restricted authorization credentials can be directly derived from macaroons, without proxying requests to the target service, or re-minting new credentials from it. Thus, macaroons can help avoid the latency and overhead, loss of transparency, and trust requirements imposed by the proxying and credential re-minting patterns commonly used in the today’s Cloud.

For example, every macaroon is likely to contain one or more validity-period caveats whose predicates restrict the time (at the target service) during which the macaroon will authorize requests; similarly, each macaroon is likely to have caveats that restrict the arguments permitted in requests. A macaroon may have multiple caveats on the same attribute, such as time, in which case all the caveats’ predicates must hold true in the context of requests.

It is straightforward to add and enforce such *first-party caveats* that confine the context observed at the target service when a macaroon is used. In the Cloud, such restrictions on bearer tokens are sometimes used to reduce the ease by which tokens can be stolen and abused by attackers, while retaining the flexibility and privacy benefits of token-based authorization [5, 13, 17, 31]. Because requests are sent directly from clients (as shown in Figure 2), target services need only check the predicates in such caveats against the context of incoming requests, for example to restrict the source IP address (or, better yet, ChannelID [21]).

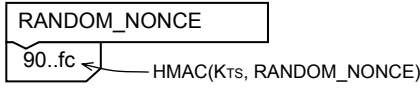
However, macaroons’ flexibility stems mostly from *third-party caveats*, their main innovation, which allow a macaroon to specify and require any number of holder-of-key proofs<sup>3</sup> to be presented with authorized requests. For example, instead of using time-based validity-period caveats, macaroons might use caveats for a third-party revocation-checking service, and require authorized requests to present fresh proofs discharging those caveats. Such third-party caveats can ensure that each request authorized by a macaroon is subject to any number of extra steps for authentication, authorization, or audit—e.g., to name just a few, checks of network time, authentication at identity providers, auditing and provenance tracking at document repositories, scanning by anti-virus engines, and vetting by a DoS-prevention and risk-analysis service [29].

Even when macaroons are used to enforce complex authorization policies, there is no particular need for end users to be aware of macaroons. In any construction, client-side macaroons will be handled and processed not by users, but by software such as client-side JavaScript in Web browsers. Thus, users can remain equally unaware of macaroons as they are of current Cloud authorization tokens.

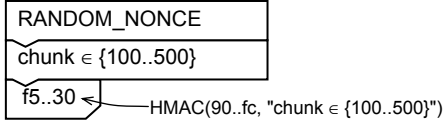
Revisiting the example at the start of this section, consider a construction where all macaroons are encoded into “unlisted URLs.” Thus, instead of a HTTP session cookie, the owner may hold a macaroon URL granting them full access to their

<sup>2</sup>Protocols like OAuth2 [17] use the term *relying party*, not target service.

<sup>3</sup>In distributed authorization, the term-of-art *holder-of-key proof* is used when a party is authenticated by proving possession of a key, e.g., by signing a protocol message with a private key [30]. Macaroons use holder-of-key proofs extensively, in particular for third-party-caveat discharges.



Above is an example of the simplest possible macaroon credential. If it grants bearers access to a key-value storage service for chunks, it can be attenuated by adding a caveat whose predicate restricts accessible chunks, as below:



To check the macaroons' integrity, the storage service can compute and verify the terminal, derived HMAC value, even when the resulting macaroon is further attenuated with a caveat whose predicate permits only reading, as below:

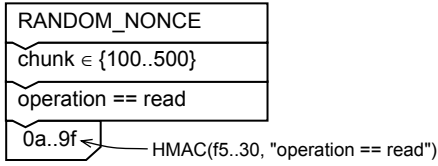


Fig. 3. Three macaroons, respectively giving full access to all chunks at a key-value storage service, permitting all requests to chunks in the range 100 to 500, or allowing only reading from that range. Starting at its key  $K_{TS}$ , the service can verify the integrity of each macaroon by deriving its chain of nested HMAC values. Macaroons are presented with requests, and the service permits only requests that satisfy the macaroons' caveats.

private images at  $TS$ . From this URL, the owner can create a new, derived macaroon URL for read-only access to an image, embed within it a third-party caveat that requires proof of membership in a group  $G$  at service  $A$ , and share it via email. Any recipient of this derived URL would have to get a fresh discharge for the third-party caveat—which may require the user to log into  $A$ , via a redirection—and pass it along to  $TS$  when using the URL to view the image. All of the above could be accomplished using software, such that both parties remained unaware of the use of macaroons.

### III. CONCRETE MECHANISMS AND EXAMPLES

Concretely, a macaroon granting authority to a Cloud server is constructed from a sequence of messages and a chain of keyed cryptographic digests derived from those messages. This chain of digest values is computed using HMAC functions with appropriate truncation (e.g., to 128 bits) [43]. Each macaroon contains, however, only the terminal, final HMAC value in the chain, with this value serving as the macaroon's *signature*: a generalized message authentication code that (i) allows target services to verify the integrity of macaroons, and (ii) also allows shared keys to be established with the bearers of any derived macaroons.

Fundamental to macaroons is how this chain of keyed HMAC values is constructed, in a nested fashion, along the sequence of messages in a macaroon—of which, all but the first are caveats. The first message in a macaroon is required to be a public, opaque *key identifier* that maps to a secret *root key* known only to the target service. (Such key identifiers can be implemented, e.g., using random nonces, indices into a database at the target service, keyed HMACs, or using public-key or secret-key encryption; see [37, Sections 4.2 and 4.3]

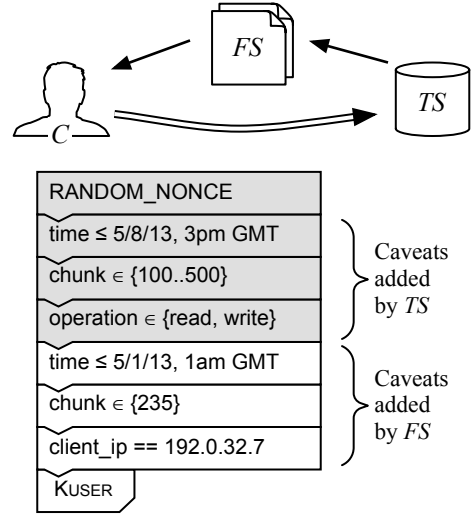


Fig. 4. A macaroon held by the user at client  $C$ , authorizing limited-time access to chunk 235 at a key-value storage service  $TS$  from one network IP address. This macaroon is derived from another (shown shaded in gray) that a social-networking “forum service”  $FS$  holds for a range of chunks at  $TS$ —delegating and attenuating the authority of that macaroon.

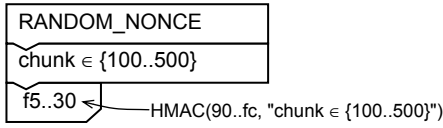
for a discussion.) Starting with this root key, a macaroon's chain of HMAC values is derived by computing, for each message, a keyed HMAC value, nested so that each such value itself is used as the HMAC key for the next message. Thus, a macaroon with two messages  $MsgId$  and  $MsgCav$  and root key  $K_R$  could have the signature  $HMAC(K_{tmp}, MsgCav)$ , where  $K_{tmp}$  is  $HMAC(K_R, MsgId)$ .

An example of this concrete macaroon construction is shown in Figure 3. (This example, like others in this paper, draws from the distributed systems literature where related chained-message-digest constructions have long been used for authorizing access to networked storage [3, 25, 51].)

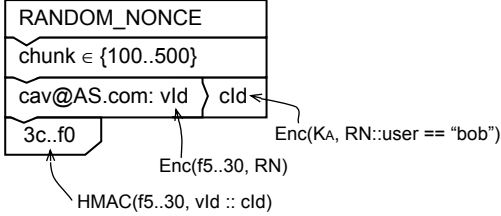
In Figure 3, starting from a macaroon with a random nonce key identifier, and a secret key  $K_{TS}$  held at the target service, two macaroons are successively derived by adding caveat predicates and chaining the HMAC signatures. Each macaroon authorizes requests to a target service that offers key-value storage for *chunks*, with caveats in the derived macaroons restricting the authorized chunks and operations. To verify the integrity of these macaroons, the target service need only check their signature against the HMAC values 90..fc, f5..30, or 0a..9f, derived using its key  $K_{TS}$ .

This construction makes it particularly simple to delegate and attenuate authorization credentials, since the chain of HMAC signatures establishes shared keys between the target service and the bearers of successive, derived macaroons.

For example, Figure 4 shows a macaroon authorizing a user at client  $C$  to access chunk 235 at a key-value storage service  $TS$ . This macaroon has been derived as a strict attenuation of another, more permissive macaroon—held by the social-networking “forum service”  $FS$ —by both eliding that macaroon's signature (called, say  $K_{FS}$ ), and by adding caveats whose predicates require a specific chunk and client IP address to be present in the request context. Even so, this macaroon's signature,  $K_{USER}$ , both allows the target storage service to verify its integrity, and also allows its bearer to



The above macaroon from the example in Figure 3 permits access to a range of chunks. By deriving a macaroon with an added third-party caveat, that authority can be delegated to a user “bob” at an authentication service, as below:



A third-party caveat message contains an root key and assertion encrypted for the discharging service (above, cld), a hint to its location (above, AS.com), and the same key encrypted for the target service to allow verification (above, vld). The signature of a discharge macaroon (below) is chained from this root key.

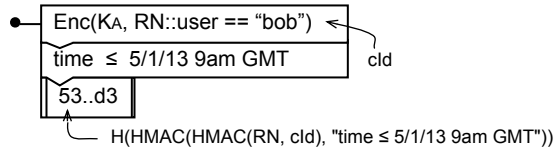


Fig. 5. A macaroon with a third-party caveat (in the middle), derived from another macaroon (at the top). At the bottom is a discharge macaroon for the caveat (whose signature is hashed to prevent further chaining), created by a third-party holder of the shared key KA after checking the assertion in the key identifier cld. By deriving the chain of nested HMAC values, the target service can verify the integrity of such discharge macaroons by decrypting the encrypted root key identifier vld.

derive new, further attenuated macaroons. In this manner, macaroon signatures both protect integrity and act as keys.

The third-party caveats in macaroons are requirements for holder-of-key proofs: assertions of one or more predicates signed by the holder of a key. The manner in which these requirements are embedded into a macaroon, and its chain of HMAC values, allows (i) the appropriate parties to create proofs asserting those predicates, (ii) those proofs, themselves, to embed additional caveats (even for new, additional third parties), and, (iii) the target service to verify that all required proofs have been discharged, recursively.

A proof that discharges a macaroon’s third-party caveat is itself a macaroon, as shown in Figure 5 (at the bottom). Such caveat discharge macaroons have a key identifier that encodes both the root key for deriving holder-of-key proofs and the predicates to be asserted by the third party (in Figure 5, it is denoted cld, and contains the key RN paired with the caveat predicate user == “bob” encrypted under the key KA). Conceptually, this key identifier can be thought of as a message encrypted with a public key for the third-party.

This same caveat key identifier is also contained in the message that a macaroon embeds for a third-party caveat, along with a hint to the location of caveat-discharging service (in Figure 5, that hint is @AS.com). This message also contains another, encrypted copy of the root key for holder-of-key proofs, encrypted with a key from the signature of

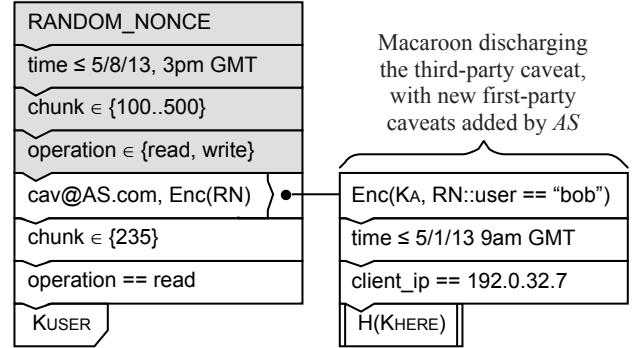
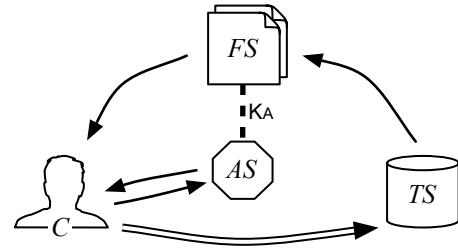


Fig. 6. Two macaroons that together give client C limited-time read access to chunk 235 at the key-value storage service TS. Using its macaroon from TS, the “forum service” FS derives for C a macaroon (on the left) that embeds a third-party caveat for an authentication service AS, confining access to clients C where a user “bob” is logged into AS. That caveat contains an encrypted root key and a key identifier. Using that key identifier, AS derives for C a macaroon (on the right) that asserts, for only a limited time, and for only one IP address, that AS has verified client C’s user.

the macaroon onto which the third-party caveat is added (in Figure 5, it is denoted vld, and is the root key RN, encrypted using the HMAC value f5..30). The target server can decrypt this second copy of the root key, and use it to verify the signatures of caveat discharge macaroons.

In general, to authorize a request, a target service may need to verify an entire set of macaroons presented with that request, and establish (i) that caveat discharge macaroons can be found for all third-party caveats, and (ii) that all first-party caveat predicates hold true in the context of the request. Target services can perform this verification by inducing a tree from the set of presented macaroons, and checking their macaroon signatures against the chain of HMAC values derived along each branch from the root to the leaves. To build this tree, the messages in the third-party caveats allow the target service to match the key identifiers of caveat discharge macaroons and decrypt their root keys—in Figure 5, respectively, the identifier Enc(KA, RN::user == “bob”) and root key Enc(f5..30, RN).

The main advantages and innovative aspects of macaroons are illustrated by the example of Figure 6, in combination. From its existing macaroon for the service TS, the forum service FS can derive a macaroon for user “bob” that attenuates *what* aspects of TS are accessible; simultaneously, FS can delegate the authentication of users to a third-party service AS, using the key KA it holds for AS. (Of course, for this, FS must know AS, and trust it to perform authentication.) The AS service can evaluate the request from client C however it chooses, before returning a macaroon that discharges the user == “bob” caveat. Notably, this caveat discharge macaroon may be very short lived (e.g., be valid for minutes, or seconds), due to the efficiency and low overhead macaroons’ HMAC-based

enough extra messages can emulate public-key-based mechanisms; therefore, these proofs are omitted here. Meanwhile, the formalization of macaroons in authorization logic has been more clarifying, and is presented in the appendix.

HMAC-based macaroons do suffer in one important way compared to public-key-based credentials: only the target service that originally mints a macaroon can check its validity. Specifically, the verification of an HMAC-based macaroon requires knowledge of its root key—and since this key confers the ability to arbitrarily modify the macaroon and its caveats, it cannot be widely shared. On the other hand, third-party verification is possible when using asymmetric signing keys and public verification keys, as in SPKI/SDSI, or the public-key-based macaroons described later in Section V-H. However, the HMAC-based macaroons are orders-of-magnitude more efficient (as shown in Section VII), which, in many cases, is a worthwhile trade-off.

Macaroons' flexibility does not obviate the need to analyze the security of any concrete authorization protocols using them—just as with any other type of authorization credential. In particular, such protocols must account for how target services learn, during verification, the intermediate signatures of macaroons, as well as the root keys of all caveat discharge macaroons, and consider how—in an adversarial model—other parties may also learn those keys, e.g., due to flaws in target services. Similarly, the concrete means for establishing macaroon key identifiers must be analyzed, especially if shared symmetric keys are used to add third-party caveats. Fortunately, many frameworks exist for the analysis of authorization protocols that use public-key, shared-key, and MAC-based constructions [45, 10, 4]. Also, such analysis can be greatly simplified if fresh, pairwise symmetric keys establish trust between principals and if macaroons use caveats with fresh, high-entropy root keys that are unique and distinct from root keys in other macaroons. For this purpose, protocols may choose to derive root keys using the entropy of intermediate macaroon signatures.

## V. IMPLEMENTATION CONCERNS

This section considers how distributed systems may implement macaroon-based authorization. The choice of topics is based on experiences building prototype macaroon mechanisms for Cloud clients, servers, and mobile devices, and using these to implement both new authorization protocols and to improve existing ones, like those based on OAuth2 [17].

Empirically, in distributed systems, macaroons are simple to implement, easy to integrate with existing code, and incur little overhead. In particular, it simplifies the task that much of macaroons' flexibility stems from the semantics of caveats being left up to application services. However, macaroon-based systems still need to support a common library of well-known caveats—such as for specifying expiration time, as well as predicates about client context such as IP addresses, ChannelID, etc.—leaving service- or application-specific caveat predicates to be used in specific protocols.

### A. Encoding Key Identifiers

The key identifiers for both macaroons and caveats may be constructed using service-specific means, as discussed in

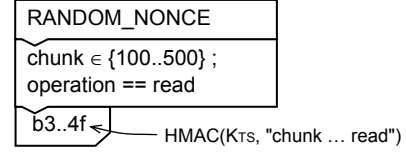


Fig. 9. Multiple caveats under one signature, as an optimization.

Section IV. For example, root keys, lists of predicates, or other secrets may be stored in a database at the service and identified by an index key (such as a random nonce, like in Section III). Such identifiers have the nice property of being completely opaque and allowing revocation to be simply done by deleting a database row; however, the downside is that the target must maintain a database, either centralized or replicated across the servers operating the service.

Alternatively, one may minimize state kept at the principal's server nodes by encrypting the keys and other secrets to be identified. For this, shared, symmetric encryption keys may be used, known to and trusted by each principal pair; alternatively, public-key encryption may be used, which can allow the same key to be used by several principals to create caveats for a single service that holds the private key.

### B. Minting Caveat Root Keys

The root keys for caveats and caveat discharge macaroons are shared across principals; in particular, the root keys of all verified macaroons become known to the target service for a request. Thus, such root keys should not be reused—e.g., by embedding the same caveat identifier in multiple macaroons—unless this is strictly necessary (e.g., for performance reasons), and it is done in a known-to-be secure fashion (e.g., verified by a separate protocol analysis). In particular, while such reuse may be safe as long as all macaroons are for the same target service, it's most likely not safe when multiple target services are involved.

Therefore, to add a third-party caveat to a macaroon, an unpredictable caveat root key must be chosen—otherwise malicious parties might guess the root key and forge a discharge macaroon—even though some principals (e.g., Web browser frames, or embedded devices), may not have a good source of entropy for creating truly random keys.

To address this concern, caveat root keys may be derived from the signature of the embedding macaroon. Concretely, for adding a caveat to a macaroon with signature  $sig$ , its bearer may generate a strong root key using  $HMAC(sig, n)$ —where  $n$  is an arbitrary nonce—because HMAC is a pseudo-random function [43]. Such a key will be unique, due to the second pre-image resistance of HMAC, and will therefore never be repeated across different caveats.

### C. Optimizations and Privacy

Macaroons permit many optimizations, in both their primitives and applications. For example, a reduced number of operations can be used for chained-MACs, if caveats are combined as in Figure 9.

Both as an optimization, and also to protect privacy, applications and services may choose to omit certain information from caveat discharge macaroons. In particular, for nested

third-party caveats, an inner, nested discharge macaroon need not be included in the discharge for the outer caveat. Thus, for example, if a third party caveat may restrict access to users within a certain age range, the caveat discharging service may add a caveat that requires a third-party authority to assert the bearer’s date of birth; subsequently, the same discharging service may remove this birthdate-assertion caveat when it issues a discharge macaroon, to not reveal the bearer’s age, thereby protecting their privacy.

With regards to privacy, it should be noted that attackers may try to add third-party caveats to probe for private information—e.g., by tricking users to reveal membership in some group. However, for this, attackers must be able to both trigger users to make requests with certain parameters, and observe the results. Under these conditions, similar attacks are equally possible in many existing authorization frameworks (e.g., SPKI/SDSI), and not specific to macaroons.

#### D. Local Discharging of Third-party Caveats

Third-party caveats can be used to implement decentralized authorization using holder-of-key proofs from authentication servers, as explained in earlier examples. However, third-party caveats may be discharged not just by networked servers, but by any isolated protection domain capable of holding secrets—such as, for example, a Web browser frame or extension, a mobile application, or a hardware token that is available locally to a user at a client.

The delegation of third-party caveat discharging to such local principals can improve both performance (by reducing network messages), as well as the precision of authorization policy enforcement. For example, a collection of unrelated Web services may use the same federated login system; if each service maintains separate sessions, logging out of one service may leave the user, unknowingly, still logged in and authorized in other services, and thereby put them at risk. Instead, using macaroons, each request, on all sessions, may derive a fresh, or very short-lived, caveat discharge from the third-party federated login service (e.g., via a hidden `iframe` that holds the caveat root key to derive discharge macaroons and is accessible from each service’s Web page). Thus, a federated login service may hold the authoritative credentials, and services may be decoupled, yet the user need log out only once to immediately de-authorize all services.

#### E. Revocation and Versioning

As previously mentioned in Section II, macaroons excel at the common revocation strategies for credential-based authorization: using short-lived, fresh credentials, that can be subject to state-based checking and can be split between principals [48, Chapter 9.8]. Indeed, the local discharging of caveats of the previous subsection is just one example of split credentials, subject to state-based checks. For other workloads or desired policies, revocation may be done via a database “whitelist” (resp. “blacklist”) of valid macaroons (resp. invalid), via embedding expiry times or epoch counters in macaroons, or via some combination of the above.

Commonly, state-based revocation may be via a third-party caveat to a service like a network time authority. To avoid the need for redundant discharging, such caveats might—when

they encode a monotonic property—use the same caveat identifier (and, hence, the same root key), for multiple macaroons, without risk. Thus, for example, a Web browser might hold hundreds of macaroons for images at the same Cloud service, such that all macaroons share the same third-party revocation-check caveat, and a single macaroon can discharge that caveat for any image. Of course, this discharge macaroon can, and should, be very short lived.

Revocation may also be required when target services are updated. For example, the operation `delete` may be added to a target service that offers previously-immutable, read-only storage, and the existing macaroons for this target may not constrain the operations in requests. In this case, the service may implement a default-deny policy for the `delete` operation, and authorizing its use only in future macaroons. As a more principled approach, targets may *version* aspects of their service, and embed first-party predicates in macaroons that constrain the authorized versions.

#### F. Disjunctive and Negated Caveats

When adding third-party caveats to a macaroon, it may be desirable to add a set of them where only one needs to be discharged. An example use case would be to allow a client to choose which authentication provider to use. One way of achieving this would be to augment macaroons to also include a conjunction or disjunction operator at the beginning of every caveat list. The `Verify` method would then accordingly conjunct or disjunct the proof obligations specified by the caveats. This leads to a notion of caveats on a macaroon as a logical formula with both disjunction and conjunction.

One may ask if it makes sense to add negation as well, where a macaroon can only be invoked if a certain caveat is *not* discharged. However, the semantics of negation become tricky when the negated caveat discharge macaroon contains further nested third-party caveats. Thus, negation should only be supported at the level of predicate checking, e.g., along the lines of `op ∉ {write, delete}`, if needed.

#### G. Caveats on the Structure of Macaroons

In some applications, there may be a need for caveat predicates to make assertions about the structure and values in the set of authorizing macaroons. In this case, one macaroon’s caveats could restrict the shape of the tree of macaroons that is induced during verification at the target service, or the values permitted in another macaroon in that tree.

Such a *structural caveat* may, for example, be used to enforce notions of well-formed transactions, by ensuring that a certain order of nested third-party caveat discharges are present [15]. Structural caveats may also name values in other caveats or discharge macaroons. For example, the BrowserID protocol behind Mozilla Persona [42] states that identity assertions (minted by a Web browser and verified by a trusted service) must successfully verify only if they state the domain name of the relying party they are meant for. Recasting BrowserID in terms of macaroons, this check can be enforced in the form of a structural caveat.

Structural caveats can also be used to make assertions about public-key signatures. For example, a service may mint

a macaroon that requires bearers to prove possession of a certain private key. For this, the service can add a first-party structural caveat whose predicate states “the set of macaroons must contain a digitally signed message  $m$ , verifiable by the public key  $P$ .” The bearer would then be required to present such a signed message along with each request to the target service, if it is to be authorized by a macaroon with this caveat.

#### H. Public-key-based Macaroon Constructions

Many existing authorization systems are based on public-key certificates, including SPKI/SDSI [16], KeyNote trust management [11], etc. Several of these systems are not truly decentralized, but instead require a globally-accessible infrastructure, such as a revocation list, or a repository of certificates. Further, upon an authorization request, some of these systems carry out a certificate-chain discovery process that can be costly; e.g., the worst-case complexity for SPKI/SDSI being cubic in the size of the repository, as shown in [16].

Instead, authorization based on public-key certificates can benefit from following the patterns of macaroon constructions, as an instance of proof-carrying authorization [6]. Already, some public-key-based applications use a delegation pattern where authority is sent between principals by chaining an existing certificate to a newly-minted public/private key pair—handed off to a new principal—such that the recipient holds a private key whose public key is certified to wield a part of the sender’s authority.

By having each principal, recursively, apply the above pattern to all authorizations, public-key certificates can effect the same type of delegation, attenuation, and key distribution as macaroons. While increasing the number of online messages, following such a macaroon-based pattern eliminates the need to maintain a global certificate repository, since each principal will individually manage the set of certificates relevant to it, and each certificate is unique.

Furthermore, by appropriately choosing the certificate validity times, and the predicates in their assertions, other benefits of macaroons may be replicated. In particular, such a certificate-based construction can provide the same privacy benefits as macaroons, in preventing linkability. Unfortunately, unlike the MAC in macaroons, the cryptographic primitives of public-key certificates can incur significant overhead—as shown in the measurements of Section VII—especially for the minting of the new public/private key pairs essential to the above construction. Therefore, to be more widely applicable, public-key-based macaroon constructions might also make use of HMAC-based signatures, e.g., as in [41].

## VI. APPLICATIONS AND USE PATTERNS

Macaroons may be used not only to reimplement existing authorization mechanisms—to state them using a single, unified model, rather than using specific, bespoke protocols—but also to enable new types of authorization for Cloud applications. For example, as in Section V-D, macaroon credentials may be split, so that a separate party holds a necessary caveat discharge, thereby enabling a form of privilege amplification similar to that in [52]. In addition to building such new protocols, macaroons can also be used to enhance several existing authorization mechanisms.

#### A. Macaroons and End Users

Macaroons are useful for many existing end-user Cloud application scenarios. For example, a macaroon can be minted for sharing an image on Google Docs or Dropbox, with third-party caveats limiting its use to a particular set of users. In such scenarios, when making use of a macaroon, end users need not experience any significant change from the workflow they use today. The client interface of the Cloud service could allow the user to request an “unlisted URL”, with options for selecting the allowed recipients, and provide means for that URL to be shared with those parties, e.g., over email.

Sometimes, a user’s management of macaroon credentials may involve more than one application. For example, when sharing a Dropbox image with a limited group, a user might copy its URL from Dropbox to a service such as Google+ or Facebook, and use that service’s client interface to pick the recipient group and attenuate the authority of the URL’s macaroon credentials. Similarly, when visiting the link for this shared URL, its recipients might be redirected to a login screen, to discharge third-party caveats, before being able to access the image at the URL.

In general, since macaroons are easily serialized, the current mechanisms for sharing and storing access tokens over email, social postings, and chat may continue to work as before. However, instead of copying and pasting URLs, the user interfaces for macaroons may be more user-friendly, e.g., taking the form of explicit, custom “Share” buttons, similar to those used for minting OAuth2 tokens today.

#### B. Existing Use of Contextual Caveats

Current bearer token authorization relies heavily on implementation guidelines, which puts the security of the mechanisms in the hands of implementors. For example, Mozilla’s BrowserID recommends constraining messages by checking the domain of the relying party [42], and many similar checks apply to OAuth2 [40]. By using macaroons, such guidelines can be made into explicit, mandatory contextual caveats.

As another example, ChannelID binds bearer tokens to a channel-specific public key [7]. ChannelID is well suited for use in first-party caveat predicates that aim to contextually confine the use of macaroons to specific (recipient) principals.

Such caveats are likely to use application-specific predicate languages, similar to how Bucket Policies in the Amazon S3 service define a rich policy language for contextual authorization of clients accessing S3 storage [5]. Of course, using macaroons, such policies could be set per issued token, instead of per storage bucket, efficiently and with very little effort.

#### C. Cookies as Macaroons

Cookies are widely used for authorization bearer tokens, such as HTTP session identifiers. With macaroon-based cookies, immediate benefits may be gained: macaroons would ensure integrity despite not maintaining state at the server, and, through the use of caveats, allow the cookies to be bound to specific, observable client context, such as IP address, user agent strings, referrers etc. An interesting benefit is also that clients could add caveats, for example to limit the usefulness of stolen cookies. Even simple attenuation might offer clients